

</>

The Anuj DECODED

</>

```
01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
};
```



```
01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver
```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

836 · Rectangle Overlap

LeetCode · Daily Question

Easy

Geometry

Separating Axis

1. What Type of Question Is This?

This is a **2-D geometry / interval-intersection** problem. The key realization is that a 2-D rectangle overlap question can always be split into **two independent 1-D interval overlap questions** — one along the x-axis, one along the y-axis. Two rectangles overlap with positive area **exactly when** their x-projections overlap *and* their y-projections overlap. This "split into independent axes" trick (sometimes called the **separating axis idea**) shows up constantly in axis-aligned geometry problems.

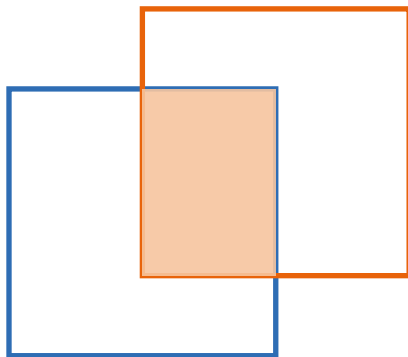
2. What Is Given?

- Two axis-aligned rectangles, **rec1** and **rec2**, each given as $[x1, y1, x2, y2]$ — $(x1, y1)$ is the bottom-left corner, $(x2, y2)$ is the top-right corner.
- Rectangle edges are always parallel to the axes (no rotation to worry about).

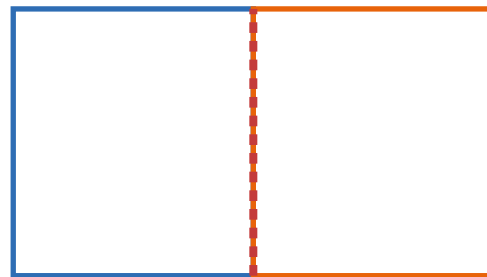
3. What Do We Need to Find?

Return **true** if the two rectangles overlap with **positive area**, and **false** otherwise. Rectangles that only touch along a shared edge or at a single shared corner do **not** count as overlapping — see the diagram below.

"Positive area" is the whole rule



Overlapping — counts



Touching only — does NOT count

Figure 1 — "overlap" means positive shared area; merely touching an edge or corner does not count.

Function Signature

■ C++

```
class Solution {
public:
    bool isRectangleOverlap(vector<int>& rec1, vector<int>& rec2) {

    }
};
```

Examples Walkthrough

Example 1 — used as the running example for every code sample below

```
Input:  rec1 = [0,0,2,2], rec2 = [1,1,3,3]
Output: true
```

rec1 spans $x \in [0,2]$, $y \in [0,2]$. rec2 spans $x \in [1,3]$, $y \in [1,3]$. Their x-ranges overlap on $[1,2]$ (positive width), and their y-ranges overlap on $[1,2]$ (positive height) — both projections overlap, so the rectangles genuinely overlap → **true**.

Example 2 — share only an edge

```
Input:  rec1 = [0,0,1,1], rec2 = [1,0,2,1]
Output: false
```

rec1 spans $x \in [0,1]$; rec2 spans $x \in [1,2]$. Their x-ranges only touch at the single point $x=1$ — zero width, not a positive overlap. Even though the y-ranges match exactly, the x-projection fails, so the rectangles don't overlap → **false**.

Example 3 — no shared region at all

```
Input:  rec1 = [0,0,1,1], rec2 = [2,2,3,3]
Output: false
```

Neither the x-ranges ($[0,1]$ vs $[2,3]$) nor the y-ranges ($[0,1]$ vs $[2,3]$) overlap at all → **false**.

Constraints

- `rec1.length == rec2.length == 4`
- $-10^9 \leq \text{rec1}[i], \text{rec2}[i] \leq 10^9$
- `rec1` and `rec2` represent a valid rectangle with a non-zero area.

Coordinates can be huge (up to a billion in magnitude), which rules out any approach that tries to iterate over individual unit squares — the solution has to work purely with the four corner coordinates, in $O(1)$ time.

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — Split the 2-D question into two 1-D questions.** A rectangle is just an x-interval combined with a y-interval. Two rectangles overlap (positive area) if and only if their x-intervals overlap (positive width) **and** their y-intervals overlap (positive height). If either axis fails to overlap, there's a vertical or horizontal line that completely separates the two rectangles — that's the "separating axis" idea.
- **Step 2 — Recall how two 1-D intervals overlap.** Given $[a1, a2]$ and $[b1, b2]$, their intersection is $[\max(a1, b1), \min(a2, b2)]$. That intersection has **positive length** exactly when $\max(a1, b1) < \min(a2, b2)$ — strictly less than, since a shared single point (equal boundary) must not count.
- **Step 3 — Apply that rule on each axis separately.** $xOverlap = \min(rec1.x2, rec2.x2) - \max(rec1.x1, rec2.x1)$; $yOverlap = \min(rec1.y2, rec2.y2) - \max(rec1.y1, rec2.y1)$. Each one is exactly the width (or height) of the shared region on that axis, using the interval-intersection formula from Step 2.
- **Step 4 — Combine both checks.** The rectangles overlap with positive area if and only if *both* $xOverlap > 0$ and $yOverlap > 0$.

Visualizing the Check

Both projections must overlap for the rectangles to overlap

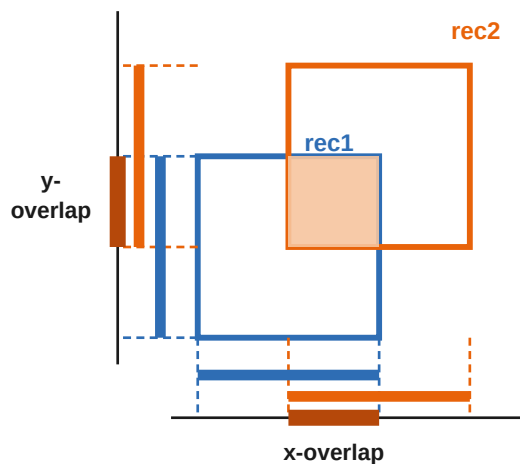


Figure 2 — Example 1's rectangles projected onto each axis. The thick highlighted segments are where the projections overlap; both must be positive-length for the rectangles to truly overlap.

Algorithm Flow

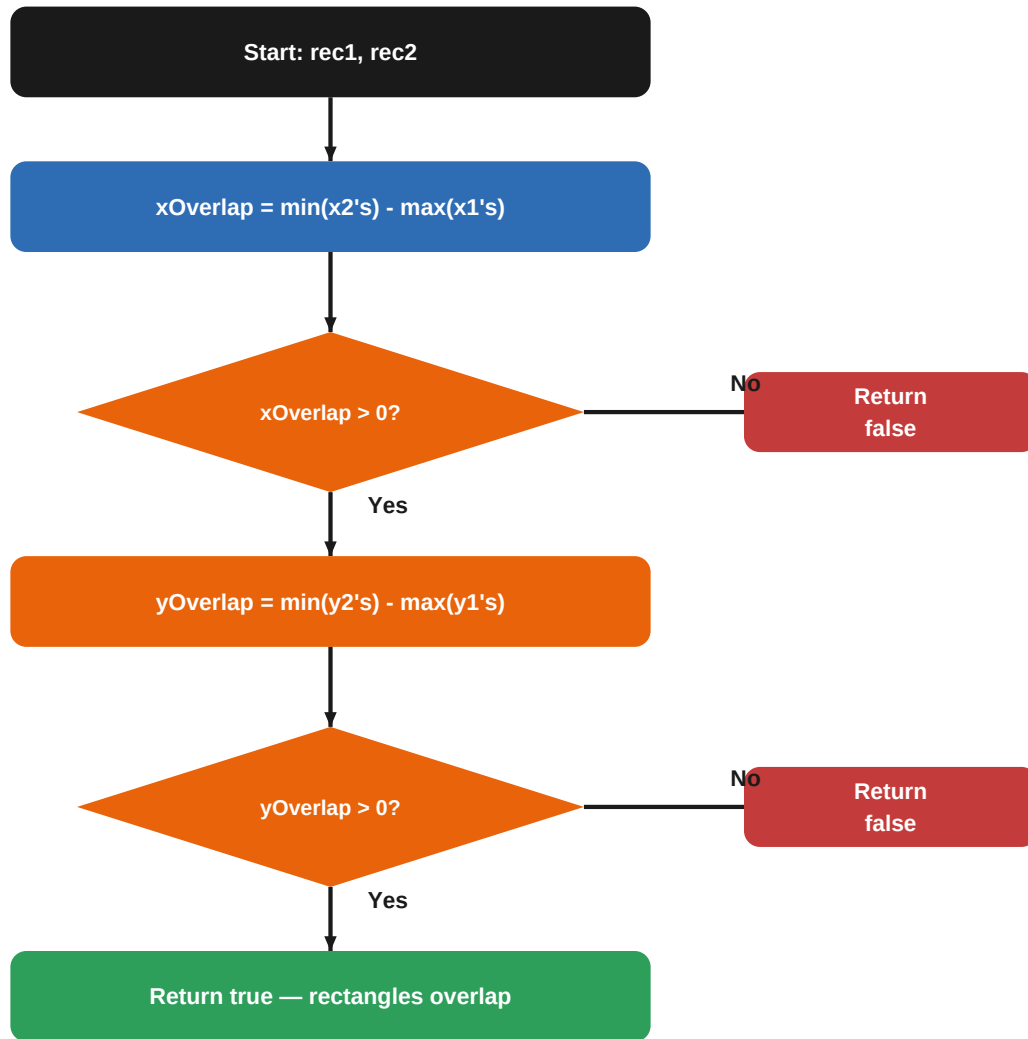


Figure 3 — the full decision flow: compute each axis's overlap, and short-circuit to false the moment either one fails.

Which Algorithm Is Best?

The direct interval-intersection check from Steps 2–4 is already $O(1)$ — there is no larger input to scale with here, only four fixed coordinates per rectangle. The three tiers below aren't really about asymptotic complexity (all three end up $O(1)$); they're about the *path* to the clean formula — starting from a case-by-case, easy-to-get-wrong enumeration, then an explicit "compute the intersection rectangle" version, and finally the minimal one-line separating-axis form.

Approach 1 — Brute Force (Case-by-Case: Corner Containment Plus a Crossing Check)

Idea: A very natural first instinct: two rectangles should overlap if a corner of one lands strictly inside the other. But that alone misses a real case — a thin, tall rectangle can pass straight through a short, wide one without any of either rectangle's *corners* ever landing inside the other. That forces a second, separate "crossing" check bolted on afterward — exactly the kind of easy-to-miss edge case that makes case-by-case geometry reasoning risky.

Steps

- Check whether any of rec1's four corners lies strictly inside rec2 (strictly, so boundary touches don't count).
- Check whether any of rec2's four corners lies strictly inside rec1.
- If either check finds a contained corner, the rectangles overlap.
- Otherwise, separately check the two possible "crossing" configurations (rec1 straddling rec2 horizontally while rec2 straddles rec1 vertically, or vice versa) — the case corner-containment alone cannot detect.
- Return true if any of these checks succeed, false otherwise.

Approach 1 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool pointInside(vector<int>& rec, int x, int y) {
        // strictly inside (not on the boundary), matching the "positive area" rule
        return x > rec[0] && x < rec[2] && y > rec[1] && y < rec[3];
    }

    bool isRectangleOverlap(vector<int>& rec1, vector<int>& rec2) {
        // Case-by-case: does any corner of one rectangle land strictly inside the other?
        vector<pair<int,int>> corners1 = {{rec1[0],rec1[1]}, {rec1[0],rec1[3]},
                                         {rec1[2],rec1[1]}, {rec1[2],rec1[3]}};
        vector<pair<int,int>> corners2 = {{rec2[0],rec2[1]}, {rec2[0],rec2[3]},
                                         {rec2[2],rec2[1]}, {rec2[2],rec2[3]}};
        for (auto& [x, y] : corners1) if (pointInside(rec2, x, y)) return true;
        for (auto& [x, y] : corners2) if (pointInside(rec1, x, y)) return true;

        // Corner-containment misses the "crossing" case (a thin tall rectangle punching
        // through a short wide one) — that needs a separate explicit check.
        bool xStraddles = rec1[0] < rec2[0] && rec2[2] < rec1[2] &&
                         rec2[1] < rec1[3] && rec1[1] < rec2[3];
        bool yStraddles = rec2[0] < rec1[0] && rec1[2] < rec2[2] &&
                         rec1[1] < rec2[3] && rec2[1] < rec1[3];
        return xStraddles || yStraddles;
    }
};

int main() {
    vector<int> rec1 = {0,0,2,2}, rec2 = {1,1,3,3};
    Solution sol;
    cout << (sol.isRectangleOverlap(rec1, rec2) ? "true" : "false") << endl; // Expected: true
    return 0;
}
```

■ Python

```
class Solution:
    def pointInside(self, rec, x, y):
        # strictly inside (not on the boundary), matching the "positive area" rule
```

```

        return rec[0] < x < rec[2] and rec[1] < y < rec[3]

    def isRectangleOverlap(self, rec1, rec2):
        corners1 = [(rec1[0], rec1[1]), (rec1[0], rec1[3]),
                    (rec1[2], rec1[1]), (rec1[2], rec1[3])]
        corners2 = [(rec2[0], rec2[1]), (rec2[0], rec2[3]),
                    (rec2[2], rec2[1]), (rec2[2], rec2[3])]

        for x, y in corners1:
            if self.pointInside(rec2, x, y):
                return True
        for x, y in corners2:
            if self.pointInside(rec1, x, y):
                return True

        # Corner-containment misses the "crossing" case – needs a separate explicit check.
        xStraddles = (rec1[0] < rec2[0] and rec2[2] < rec1[2] and
                     rec2[1] < rec1[3] and rec1[1] < rec2[3])
        yStraddles = (rec2[0] < rec1[0] and rec1[2] < rec2[2] and
                     rec1[1] < rec2[3] and rec2[1] < rec1[3])
        return xStraddles or yStraddles

if __name__ == "__main__":
    rec1, rec2 = [0,0,2,2], [1,1,3,3]
    print(Solution().isRectangleOverlap(rec1, rec2))    # Expected output: True

```

■ JavaScript

```

class Solution {
    pointInside(rec, x, y) {
        // strictly inside (not on the boundary), matching the "positive area" rule
        return x > rec[0] && x < rec[2] && y > rec[1] && y < rec[3];
    }

    isRectangleOverlap(rec1, rec2) {
        const corners1 = [[rec1[0],rec1[1]], [rec1[0],rec1[3]], [rec1[2],rec1[1]], [rec1[2],rec1[3]]];
        const corners2 = [[rec2[0],rec2[1]], [rec2[0],rec2[3]], [rec2[2],rec2[1]], [rec2[2],rec2[3]]];

        for (const [x, y] of corners1) if (this.pointInside(rec2, x, y)) return true;
        for (const [x, y] of corners2) if (this.pointInside(rec1, x, y)) return true;

        // Corner-containment misses the "crossing" case – needs a separate explicit check.
        const xStraddles = rec1[0] < rec2[0] && rec2[2] < rec1[2] &&
                          rec2[1] < rec1[3] && rec1[1] < rec2[3];
        const yStraddles = rec2[0] < rec1[0] && rec1[2] < rec2[2] &&
                          rec1[1] < rec2[3] && rec2[1] < rec1[3];
        return xStraddles || yStraddles;
    }
}

const rec1 = [0,0,2,2], rec2 = [1,1,3,3];
console.log(new Solution().isRectangleOverlap(rec1, rec2)); // Expected output: true

```

Complexity: Every check is $O(1)$ (a handful of coordinate comparisons), so this is technically $O(1)$ time just like the other tiers — but it needs far more branches, and it's easy to genuinely get wrong by forgetting the crossing case entirely (a bug that would silently fail on inputs like a cross/plus shape). Correctness risk, not speed, is this approach's real cost.

Approach 2 — Better (Compute the Intersection Rectangle Explicitly)

Idea: Apply the Step 2 interval-intersection formula on both axes at once: directly build the coordinates of the rectangle that *would be* the overlap region, then simply check whether that computed rectangle has a positive width and height. No case enumeration, no risk of a missed "crossing" scenario — it falls out naturally from the formula.

Steps

- Compute $ix1 = \max(\text{rec1.x1}, \text{rec2.x1})$ and $iy1 = \max(\text{rec1.y1}, \text{rec2.y1})$ — the intersection's bottom-left corner.
- Compute $ix2 = \min(\text{rec1.x2}, \text{rec2.x2})$ and $iy2 = \min(\text{rec1.y2}, \text{rec2.y2})$ — the intersection's top-right corner.
- $\text{width} = ix2 - ix1$, $\text{height} = iy2 - iy1$.
- Return true only if both width and height are strictly positive.

Approach 2 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isRectangleOverlap(vector<int>& rec1, vector<int>& rec2) {
        // Explicitly compute the intersection rectangle's boundaries.
        int ix1 = max(rec1[0], rec2[0]);
        int iy1 = max(rec1[1], rec2[1]);
        int ix2 = min(rec1[2], rec2[2]);
        int iy2 = min(rec1[3], rec2[3]);

        int width = ix2 - ix1;
        int height = iy2 - iy1;

        return width > 0 && height > 0;    // positive area required
    }
};

int main() {
    vector<int> rec1 = {0,0,2,2}, rec2 = {1,1,3,3};
    Solution sol;
    cout << (sol.isRectangleOverlap(rec1, rec2) ? "true" : "false") << endl;    // Expected: true
    return 0;
}
```

■ Python

```
class Solution:
    def isRectangleOverlap(self, rec1, rec2):
        # Explicitly compute the intersection rectangle's boundaries.
        ix1 = max(rec1[0], rec2[0])
        iy1 = max(rec1[1], rec2[1])
        ix2 = min(rec1[2], rec2[2])
        iy2 = min(rec1[3], rec2[3])

        width = ix2 - ix1
        height = iy2 - iy1

        return width > 0 and height > 0    # positive area required

if __name__ == "__main__":
    rec1, rec2 = [0,0,2,2], [1,1,3,3]
    print(Solution().isRectangleOverlap(rec1, rec2))    # Expected output: True
```

```
class Solution {
  isRectangleOverlap(rec1, rec2) {
    // Explicitly compute the intersection rectangle's boundaries.
    const ix1 = Math.max(rec1[0], rec2[0]);
    const iy1 = Math.max(rec1[1], rec2[1]);
    const ix2 = Math.min(rec1[2], rec2[2]);
    const iy2 = Math.min(rec1[3], rec2[3]);

    const width = ix2 - ix1;
    const height = iy2 - iy1;

    return width > 0 && height > 0; // positive area required
  }
}

const rec1 = [0,0,2,2], rec2 = [1,1,3,3];
console.log(new Solution().isRectangleOverlap(rec1, rec2)); // Expected output: true
```

Complexity: Still $O(1)$ — four max/min operations and two subtractions, no branching beyond the final positive-area check. Conceptually cleaner and far less bug-prone than Approach 1, since it handles every configuration (corner containment, edge crossing, one rectangle fully containing the other) uniformly through the same formula.

Approach 3 — Optimal (Negate the "Separated" Condition — One Line)

Idea: Instead of computing an intersection and checking it's positive, directly ask the opposite question: *when do the rectangles definitely NOT overlap?* That happens exactly when one rectangle is entirely to one side of the other along some axis — four simple cases. The rectangles overlap precisely when **none** of those four separation cases hold, which is just De Morgan's law applied to Approach 2's positive-width-and-height check.

Steps

- Rectangles are separated (cannot overlap) if rec1 is entirely to the left of rec2 ($\text{rec1.x2} \leq \text{rec2.x1}$), or rec2 is entirely to the left of rec1, or rec1 is entirely below rec2 ($\text{rec1.y2} \leq \text{rec2.y1}$), or rec2 is entirely below rec1.
- Combine all four separation conditions with OR.
- Return the logical negation of that combined condition — true exactly when none of the four separations hold.

Approach 3 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isRectangleOverlap(vector<int>& rec1, vector<int>& rec2) {
        // Overlap fails only if the rectangles are separated along the x-axis or the y-axis.
        return !(rec1[2] <= rec2[0] || // rec1 entirely left of rec2
                rec2[2] <= rec1[0] || // rec2 entirely left of rec1
                rec1[3] <= rec2[1] || // rec1 entirely below rec2
                rec2[3] <= rec1[1]); // rec2 entirely below rec1
    }
};

int main() {
    vector<int> rec1 = {0,0,2,2}, rec2 = {1,1,3,3};
    Solution sol;
    cout << (sol.isRectangleOverlap(rec1, rec2) ? "true" : "false") << endl; // Expected: true
    return 0;
}
```

■ Python

```
class Solution:
    def isRectangleOverlap(self, rec1, rec2):
        # Overlap fails only if the rectangles are separated along the x-axis or the y-axis.
        return not (
            rec1[2] <= rec2[0] or # rec1 entirely left of rec2
            rec2[2] <= rec1[0] or # rec2 entirely left of rec1
            rec1[3] <= rec2[1] or # rec1 entirely below rec2
            rec2[3] <= rec1[1]    # rec2 entirely below rec1
        )

if __name__ == "__main__":
    rec1, rec2 = [0,0,2,2], [1,1,3,3]
    print(Solution().isRectangleOverlap(rec1, rec2)) # Expected output: True
```

■ JavaScript

```
class Solution {
    isRectangleOverlap(rec1, rec2) {
```

```

    // Overlap fails only if the rectangles are separated along the x-axis or the y-axis.
    return !(
        rec1[2] <= rec2[0] || // rec1 entirely left of rec2
        rec2[2] <= rec1[0] || // rec2 entirely left of rec1
        rec1[3] <= rec2[1] || // rec1 entirely below rec2
        rec2[3] <= rec1[1]    // rec2 entirely below rec1
    );
}

const rec1 = [0,0,2,2], rec2 = [1,1,3,3];
console.log(new Solution().isRectangleOverlap(rec1, rec2)); // Expected output: true

```

Complexity: $O(1)$ time and space, identical to Approach 2 in raw operation count, but expressed as a single return statement with no intermediate variables. This is the same logic as Approach 2 (they're mathematically equivalent via De Morgan's law) — included here as the most compact, idiomatic way to write the final check once you're confident in the underlying reasoning.

Side-by-Side Comparison

1. Brute Force	Corner containment plus a separately bolted-on crossing check	Check	Easy to under-cover cases (e.g. forget cross
2. Better	Explicitly compute the intersection rectangle, check position area	Area	Handles every configuration uniformly
3. Optimal	Negate the four "definitely separated" conditions (De Morgan's law)	Or	Same guarantees as Approach 2, most com

Key Takeaways to Remember

- **Axis-aligned geometry problems almost always decompose into independent x-axis and y-axis interval problems** — look for that split before reaching for case-by-case corner or edge checks.
- The interval-intersection formula — $[\max(a_1, b_1), \min(a_2, b_2)]$ — is worth memorizing outright; it turns "do these ranges overlap" into two comparisons with no special cases.
- Use **strict** inequalities ($<$, $>$) whenever a problem specifically excludes boundary-only contact ("positive area", "do not overlap at a single point", etc.) — a single $\leq$ vs $<$ slip is the most common bug in these problems.
- "Prove the opposite is impossible" (De Morgan's law: $\text{NOT}(\text{any separation}) = \text{overlap}$) is often the cleanest way to state a geometric overlap condition, once you're sure you've enumerated every way the shapes could fail to overlap.
- All three tiers were validated against **Example 1** ($\text{rec1} = [0,0,2,2]$, $\text{rec2} = [1,1,3,3] \rightarrow \text{true}$) across C++, Python, and JavaScript, and checked against Examples 2 and 3 (both $\rightarrow \text{false}$) as well.